One-Time Memories Secure against Depth-Bounded Quantum Circuits

Kyosuke Sekii and Takashi Nishide

University of Tsukuba, Ibaraki 305-8577, Japan
s2420537@u.tsukuba.ac.jp, nishide@risk.tsukuba.ac.jp

Abstract. A one-time memory (OTM) is a useful cryptographic primitive, classically modeled after a non-interactive oblivious transfer. It is well known that secure OTMs (and more generally one-time deterministic programs) cannot exist in the standard model in either the classical or quantum setting due to Broadbent et al. (CRYPTO'13). Broadbent et al. circumvented this impossibility by assuming the existence of hardware tokens that cannot be queried in superposition. In this work, we take a different approach. Building on Liu's assumption (ITCS'23) that adversaries are limited to depth-bounded quantum circuits, we present two OTM constructions. The first is efficiently realizable and secure against adversaries restricted to constant-depth quantum circuits. The second is a feasibility result that achieves security against adversaries limited to $\mathcal{O}(\lambda^\gamma)$-depth quantum circuits by ensuring that a successful attack would necessarily require deeper quantum computations, where λ^γ is a polynomial in the security parameter λ. Our results therefore extend prior work, which either relied on hardware assumptions or considered only constant-depth-bounded adversaries. As a result, by combining our proposed quantum OTMs with the framework of Broadbent et al. (CRYPTO'13), one can also realize quantum one-time programs (OTPs) for deterministic programs.

Keywords: one-time memory · quantum cryptography.

1 Introduction

[1] In 2008, Goldwasser, Kalai, and Rothblum introduced the novel concept called the one-time program (OTP), based on the notion of a one-time memory (OTM) [16]. A (stateful) OTM in [16] is a memory device that stores two messages m_0 and m_1 and operates as follows:

1. The sender stores two messages m_0 and m_1 in the OTM token, and sets a tamper-proof bit $\beta = 0$.

[1] This manuscript is an author-prepared version of the work, and the final published version (the version of record) appears in Kyosuke Sekii and Takashi Nishide, "One-Time Memories Secure against Depth-Bounded Quantum Circuits," Indocrypt 2025, LNCS 16372, pp.365–389, Springer, 2025, available at https://doi.org/10.1007/978-3-032-13301-4_16.

2. The sender sends the token to the receiver.
3. The receiver provides an input $b \in \{0, 1\}$ to the token.
4. If $\beta = 0$, the OTM sets $\beta = 1$, and outputs m_b. If $\beta \neq 0$, it performs no operation, and outputs $\perp$.

An OTM is a memory device that permits access to exactly one of the two stored messages, m_0 or m_1, while irrevocably preventing access to the other[2]. Quantum-resistant OTMs, in particular, enable the construction of quantum-resistant OTPs [9]. OTPs allow the execution of arbitrary circuits while ensuring that, once an input is chosen and executed, the program cannot be reused with a different input. They support applications such as software copy protection and electronic money that prevents double-spending. OTMs can be implemented using hardware such as Trusted Execution Environments (TEEs) [15,33]. However, they cannot be realized without such specialized hardware in the classical settings. Since software-only implementations can be duplicated, they allow adversaries to execute the OTM on different inputs and obtain both stored values.

To address this limitation, quantum states have been considered for their intrinsic non-cloneable property [30], suggesting the possibility of software-only OTMs. However, it has been shown that even with quantum states alone, constructing OTMs, or more generally, transforming deterministic programs (including OTMs) into OTPs, remains impossible when adversaries can perform quantum computations of arbitrary depth [9,8].

These impossibility results motivate the central goal of this work: to construct OTMs, and more generally OTPs for deterministic programs, under the assumption that adversaries are limited to quantum computations of bounded depth [20]. While this is a stronger assumption beyond the standard model, it serves as a meaningful framework in that it makes relatively efficient constructions achievable and secure even on quantum computers operating in imperfect form.

1.1 Related Work

Broadbent et al. [8] constructed secure quantum OTMs using Wiesner's quantum states [29] under the assumption of stateless hardware tokens that hide internal information and disallow superposition queries. Behera et al. [5] subsequently reduced the token's query complexity and achieved noise tolerance against measurement errors. However, these tokens require more sophisticated functionality than the OTMs of [16] and still require physical delivery to the receivers, which limits their practicality.

Liu demonstrated that OTMs can be constructed from Wiesner states under the assumption of depth-bounded quantum adversaries and relatively short decoherence times of quantum states [20]. Since maintaining very deep quantum circuits is considered infeasible (NIST estimates a feasible depth of $2^{40} \sim 2^{96}$ [22]), the depth-bounded setting appears reasonable. The construction embeds

[2] This realizes the functionality of non-interactive oblivious transfer.

the basis information θ into a quantum-resistant Time-Lock Puzzle (TLP) [7]. In [20], it is assumed that the Wiesner state decays before θ can be recovered from the TLP, so the receiver is forced to fix the choice of $b \in \{0,1\}$ in advance and can obtain only one of m_0 or m_1, with retrieval deferred until the TLP deadline. For the security, the deadline must exceed the decoherence time of the quantum states, which may be on the order of an hour or longer [27].

Stambler [25] proposed an OTM construction secure against adversaries restricted to depth-bounded, non-adaptive, and geometrically local quantum operations.[3] The scheme achieves information-theoretic security in the classical sense, as no restriction is placed on the classical computational power of adversaries. However, it relies on quantum random access codes whose decoding requires exponential time, limiting its practicality. In contrast, our method achieves the computationally secure OTMs while requiring only polynomial-time classical computation, making it suitable for practical deployment.

Further Related Work: OTP for Probabilistic Program Recent works [18,19] construct OTPs from one-time probabilistic signatures [6,13] by restricting attention to probabilistic programs $f(x;r)$, where the randomness $r = \mathcal{RO}(x,\sigma)$ is derived from a valid signature σ on x and the random oracle $\mathcal{RO}$, circumventing the impossibility of transforming deterministic programs into OTPs [8,9]. In contrast, our work follows the depth-bounded adversary model [20], which enables the construction of OTPs even for deterministic programs, as shown in [9].

1.2 Related Impossibility Result: *Rewinding Attack*

As noted earlier, constructing an OTM secure against quantum polynomial time (QPT) adversaries with arbitrary-depth quantum computation is impossible [9,8]. Such an adversary can extract both m_0 and m_1 as follows: it prepares registers initialized with the sender's state (e.g., a Wiesner state), applies the OTM unitary U_{OTM} to read m_0 while leaving the input state nearly undisturbed by the Gentle Measurement Lemma, then inverts U_{OTM} to restore the original state and repeats the process with a different initialization to recover m_1. Even if the OTM read operation is specified as a classical procedure, the adversary can, in principle, implement an equivalent OTM read operation within a quantum circuit.

However, if evaluating U_{OTM} requires a quantum circuit of depth exceeding the adversary's capability, the rewinding attack becomes infeasible. The security of OTM constructions against depth-bounded adversaries relies on precisely this intuition: adversaries are limited to d-depth quantum circuits, while the depth of U_{OTM} in such OTM constructions exceeds this bound d.

[3] Geometrically local quantum operations restrict quantum computers to applying two-qubit gates only between adjacent qubits. Non-adjacent qubits must be brought together via SWAP gates, increasing circuit depth. Stambler conjectures that this locality constraint may be unnecessary and that security may rely solely on depth limitations.

1.3 Our Contributions

We propose two constructions for OTMs.

First Construction Our first construction is an efficiently realizable OTM under the restriction that adversaries are limited to constant-depth quantum circuits. It combines Wiesner states [29], as in [8,20], with obfuscation techniques [28,12] based on the LWE assumption [23]. This approach removes the need for hardware tokens, relying only on classical communication and quantum-state transmission, and avoids the timing constraint of [20], thereby yielding a more practical and flexible construction.

As noted in [8], assuming perfect quantum measurements is unrealistic; addressing this for Wiesner states requires additional countermeasures. To this end, we design the obfuscated program so that its branching program efficiently tolerates small quantum measurement errors. Moreover, applying the "compute-and-compare" obfuscation of [28,12] directly would also require obfuscating pseudorandom generators (PRGs) as well, substantially increasing execution time. Our construction avoids obfuscating PRGs through a tailored program design, thereby achieving improved efficiency.

Second Construction Building on post-quantum indistinguishability obfuscation (iO), the second construction realizes an OTM secure against $\mathcal{O}(\lambda^\gamma)$-depth quantum circuits in the quantum random oracle model, by tuning the scheme depth parameter k so that breaking it requires circuits of depth $\Omega(\lambda^{\gamma+1})$. To the best of our knowledge, this is the first such OTM secure against $\mathcal{O}(\lambda^\gamma)$-depth quantum circuits. Its security relies on the unclonability and direct product hardness of *hidden subspace states*, a primitive widely used in quantum money schemes [1,13,11]. This second construction extends security to more powerful adversaries, serving as an important feasibility result despite its reliance on heavy machinery such as iO and error correcting codes.

Comparison with Prior Works Unlike Broadbent et al. [8], our constructions are entirely software-based and tolerate realistic quantum measurement errors without relying on hardware tokens. Compared with Liu's scheme [20], our constructions avoid the dependency on short decoherence times. Moreover, our second construction can also handle adversaries bounded by non-constant depth. Stambler's work [25] requires an exponential OTM read cost, whereas our constructions require only a polynomial cost. The work in [18,19] focus only on probabilistic programs, while our constructions support deterministic programs under the assumption of depth-bounded quantum adversaries [20], thereby circumventing the known impossibility results [9,8].

2 Preliminaries

2.1 Notations

We define the set of n integers $[n] := \{1, \ldots, n\}$. For a probability distribution or random variable X, sampling of x from X is denoted as $x \leftarrow X$. When x is

selected uniformly at random from a set X, it is specifically denoted as $x \xleftarrow{\$} X$. The security parameter is denoted by λ. A negligible function is denoted by $\mathsf{negl}(\cdot)$, and a polynomial function is denoted by $\mathsf{poly}(\cdot)$. For a bit string x, $x[i]$ denotes its i-th bit, and for a function f, $f[i]$ denotes the function that outputs only the i-th bit of f's output, where the index of the first bit is 1. When a and b are bit strings, $\mathsf{HW}(a)$ denotes the Hamming weight of a, and $\mathsf{HD}(a, b)$ denotes the Hamming distance between a and b. We denote "probabilistic polynomial time" as PPT, and "quantum polynomial time" as QPT for short.

2.2 Quantum Computation

A pure quantum state of one qubit is represented as a unit column vector $|\psi\rangle$ in a complex Hilbert space. The orthonormal basis $\{|0\rangle, |1\rangle\}$ is referred to as the computational basis, and the orthonormal basis $\left\{\frac{|0\rangle+|1\rangle}{\sqrt{2}}, \frac{|0\rangle-|1\rangle}{\sqrt{2}}\right\}$ is referred to as the Hadamard basis. For simplicity, we define $|+\rangle := \frac{|0\rangle+|1\rangle}{\sqrt{2}}$, $|-\rangle := \frac{|0\rangle-|1\rangle}{\sqrt{2}}$.

2.3 Wiesner States

Let $s = s_1 \cdots s_n \in \{0,1\}^n$ and $\theta = \theta_1 \cdots \theta_n \in \{0,1\}^n$. The n-qubit Wiesner state $|s^\theta\rangle$ is defined as $|s^\theta\rangle = |s[1]^{\theta[1]}\rangle \otimes \cdots \otimes |s[n]^{\theta[n]}\rangle$ where the i-th qubit is $|s[i]^{\theta[i]}\rangle = H^{\theta[i]}|s[i]\rangle$, H denotes the Hadamard matrix, and H^0 represents the identity matrix. For example, if $s = 0110$ and $\theta = 0011$, then $|s^\theta\rangle := |01-+\rangle$.

It is well known that, without θ, recovering all bits of s from a Wiesner state $|s^\theta\rangle$ is difficult, even by applying any quantum operation or performing measurements in any basis. It is also known that when n is sufficiently large, it is hard to duplicate a Wiesner state without the knowledge of θ (Lemma 1).

Lemma 1 (Infeasibility of Cloning Wiesner States [10]). *For any even integer $n \geq 2$, any (unbounded) quantum adversaries $(\mathcal{A}, \mathcal{B}, \mathcal{C})$,*

$$\Pr[\mathsf{CloneGame}_{\mathcal{A},\mathcal{B},\mathcal{C}}(n) = 1] \leq 0.924^n < 2^{-0.11n},$$

where $\mathsf{CloneGame}_{\mathcal{A},\mathcal{B},\mathcal{C}}(n)$ is defined as:

1. *A challenger samples $s \leftarrow \{0,1\}^n$ and $\theta \leftarrow \{0,1\}^n$ conditioned on $\mathsf{HW}(\theta) = n/2$ uniformly at random, and sends $|s^\theta\rangle$ to $\mathcal{A}$.*
2. *$\mathcal{A}$ applies a quantum channel on $|s^\theta\rangle$ and obtains ρ_{BC} over registers B and C, which are then sent to $\mathcal{B}$ and $\mathcal{C}$ respectively.*
3. *The challenger sends θ to $\mathcal{B}$ and $\mathcal{C}$.*
4. *The game outputs 1 if and only if $\mathcal{B}$ outputs s_0 and $\mathcal{C}$ outputs s_1.*

Proof. This follows from a corollary of the stronger monogamy-of-entanglement result for coset states [14], since Wiesner states are a special case of coset states.

2.4 Depth-bounded Quantum Circuits [20]

Depth-bounded quantum circuits [20] are defined under the assumption that adversaries can only evaluate quantum circuits of depth at most d. In our model, shallow quantum circuits are interleaved with full measurements and classical processing, following the framework of [20]. A full measurement acts on all qubits, including both input and output registers, leaving the internal state entirely classical. The model of depth-bounded quantum circuits is illustrated in Fig. 1.

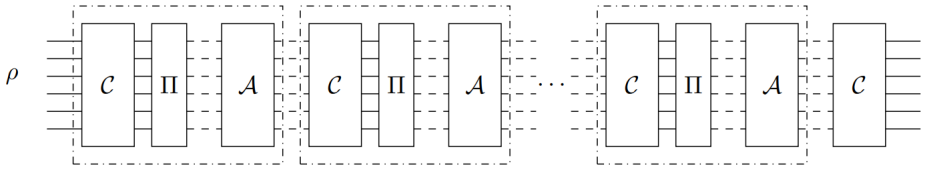


Fig. 1. d-depth bounded quantum circuits from [20]. $\mathcal{C}$ denotes quantum circuits, Π denotes measurements whereas $\mathcal{A}$ denotes classical post-processing. Dashed lines denote classical bits and solid lines denote qubits. There are polynomially many such blocks, but each $\mathcal{C}$ has depth at most d.

2.5 Quantum Random Oracle Model (QROM)

We work in the quantum random oracle model (QROM), which allows quantum circuits to access classical random oracles.

Definition 1 (Random Oracle). *A classical random oracle $\mathcal{RO}$ is defined as a unitary transformation of the form $U_f |x, y\rangle \to |x, y \oplus f(x)\rangle$ where $f : \{0,1\}^{n+1} \to \{0,1\}^m$ is a classical random function. We note that such a classical oracle can also be queried in quantum superposition.*

Unless otherwise specified, we use the term "random oracle" to mean a classical random oracle.

2.6 Measurement Error Model

In an ideal quantum computer, measuring a quantum state $|\psi\rangle$ using the projection measurement operator $\{|\psi\rangle\langle\psi|, I - |\psi\rangle\langle\psi|\}$ will always yield the outcome 0 (i.e., $|\psi\rangle$) with probability 1. However, in a quantum computer subject to a measurement error rate ϵ, the outcome of the measurement will be 0 with probability $1 - \epsilon$, and 1 with probability ϵ where ϵ is assumed to be small. In this work, we assume that when an n-qubit quantum state is measured, $n\epsilon = c$ qubits of the measurement result will be flipped, while the remaining $n - n\epsilon = n - c$ qubits are correctly measured.

2.7 One Time Memory (OTM)

An OTM scheme consists of two procedures: one for generating a token composed of quantum states and classical programs, and another for retrieving information from the token. It is parameterized by a security parameter λ and a message length ℓ_{msg}. The following definition is a variant of the one presented in [20]:

Definition 2 (Memory Token). *A memory token scheme consists of the following quantum algorithms.*

- $\mathsf{Token.Gen}(1^\lambda, m_0, m_1)$ *is a quantum algorithm that takes as input a security parameter* λ *and two messages* $m_0, m_1 \in \{0,1\}^{\ell_{msg}}$*, and outputs a quantum token* $|\mathsf{tk}\rangle$*.*
- $\mathsf{Token.Eval}(|\mathsf{tk}\rangle, b)$ *is a quantum algorithm that takes as input a quantum token* $|\mathsf{tk}\rangle$ *and a bit* $b \in \{0,1\}$*, and outputs a message* m*.*

Correctness*: For every security parameter* λ*, every message length* ℓ_{msg}*, every pair of messages* $m_0, m_1 \in \{0,1\}^{\ell_{msg}}$ *and every bit* $b \in \{0,1\}$*, we have*

$$\Pr[\mathsf{Token.Eval}(|\mathsf{tk}\rangle, b) = m_b \mid |\mathsf{tk}\rangle \leftarrow \mathsf{Token.Gen}(1^\lambda, m_0, m_1)] = 1.$$

Next, we define "one-timeness" property of OTMs. This property states that, for depth-bounded adversary, it can either learn m_0 or m_1, but not both. Then we formalize the security via the game-based definition.

Definition 3 (One-Time Memory Secure against Depth-bounded Adversary). *Consider the following game between a challenger and a depth-bounded adversary* $\mathcal{A}$*.*

$\underline{\mathsf{OTM\text{-}Game}_{\mathcal{A}}(1^\lambda):}$

1. *The challenger* $\mathcal{C}$ *samples two messages* $m_0, m_1 \in \{0,1\}^{\ell_{msg}}$*.*
2. $\mathcal{C}$ *samples* $|\mathsf{tk}\rangle \leftarrow \mathsf{Token.Gen}(1^\lambda, m_0, m_1)$ *and sends* $|\mathsf{tk}\rangle$ *to* $\mathcal{A}$*.*
3. $\mathcal{A}$ *can perform quantum computation with* $|\mathsf{tk}\rangle$*, but once the computation reaches depth* d*,* $\mathcal{A}$ *measures* $|\mathsf{tk}\rangle$ *and all quantum registers.*
4. $\mathcal{A}$ *outputs a pair* m'_0, m'_1*.*
5. $\mathcal{C}$ *outputs* 1 *if* $m'_0 = m_0$ *and* $m'_1 = m_1$*. Otherwise, it outputs* 0*.*

We say that a quantum OTM scheme $(\mathsf{Token.Gen}, \mathsf{Token.Eval})$ *is secure, if, for any depth-bounded adversary* $\mathcal{A}$*,*

$$\Pr[\mathsf{OTM\text{-}Game}_{\mathcal{A}}(1^\lambda) = 1] \leq \mathsf{negl}(\lambda)$$

We also require that both $\mathsf{Token.Gen}$ and $\mathsf{Token.Eval}$ be computable by a d-depth quantum algorithm where $d = \mathcal{O}(1)$ or $d = \mathcal{O}(\lambda^\gamma)$ in our setting.

We recall a one-time memory in the classical stateless hardware model [8,5]. The classical hardware model assumes a device that completely hides all partial information about the hardcoded program and accepts only classical inputs.

Algorithm 1 PHard$(b,\ sig)$

Hardcoded: $s,\ \theta \in \{0,1\}^n,\ m_0,\ m_1 \in \{0,1\}^{\ell_{msg}}$

$\mathsf{cnt} = 0$
for $i \in [n]$ **do**
  if $sig[i] \neq s[i] \wedge \theta[i] = b$ **then**
    $\mathsf{cnt} \leftarrow \mathsf{cnt} + 1$
  end if
end for
if $\mathsf{cnt} \leq n\epsilon$ **then**
  return m_b
else
  return $\perp$
end if

Theorem 1 (OTM in the Classical Hardware Model [5]). *Construction 4 in the stateless hardware model, which is based on Wiesner states, is an OTM protocol secure against every QPT adversary $\mathcal{A}$ making a polynomial number of classical queries to the hardware token. Moreover, Construction 4 is tolerant to a measurement error rate of up to $\epsilon = 0.14$.*

Construction 4 (OTM Construction with Classical Hardware [5]).
$\mathsf{Token.Gen}(1^\lambda, m_0, m_1)$: The sender $\mathcal{S}$ performs the following steps:

1. Sample $\theta, s \leftarrow \{0,1\}^n$ such that $\mathsf{HW}(\theta) = \frac{n}{2}$.
2. Prepare the Wiesner state $|s^\theta\rangle = H^\theta |s\rangle$.
3. Set $\epsilon \leftarrow \mathbb{R}$ where $0 \leq \epsilon \leq 0.14$.
4. Implement PHard (Algorithm 1) in the classical stateless hardware.
5. Send $|\mathsf{tk}\rangle := (|s^\theta\rangle, \mathsf{PHard})$ to the receiver.

$\mathsf{Token.Eval}(b, |\mathsf{tk}\rangle)$: Upon receiving the token $|\mathsf{tk}\rangle$ from the sender $\mathcal{S}$, the receiver $\mathcal{R}$ selects an input bit $b \in \{0,1\}$ and executes the following:

1. Parse $|\mathsf{tk}\rangle = (|s^\theta\rangle, \mathsf{PHard})$.
2. Measure $(H^b)^{\otimes n} |s^\theta\rangle$ in the computational basis to obtain s'.
3. Output $m_b \leftarrow \mathsf{PHard}(b, s')$.

2.8 Indistinguishability Obfuscation

We recall the notion of indistinguishability obfuscation (iO).

Definition 5. *An indistinguishability obfuscation scheme iO for a class of circuits $\{\mathcal{C}_\lambda\}_{\lambda\in\mathbb{N}}$ satisfies the following.*

Correctness *For all $\lambda, C \in \mathcal{C}_\lambda$, and input x,*

$$\Pr[\tilde{C}(x) = C(x) \mid \tilde{C} \leftarrow \mathsf{iO}(1^\lambda, C)] \geq 1 - \mathsf{negl}(\lambda).$$

Security *Let $C_0, C_1 \in \mathcal{C}_\lambda$ be two circuits of the same size such that $C_0(x) = C_1(x)$ for all inputs x. Then, for any QPT algorithm $\mathcal{D}$,*

$$\left|\Pr[\mathcal{D}(\tilde{C}_0, \mathsf{aux}) = 1] - \Pr[\mathcal{D}(\tilde{C}_1, \mathsf{aux}) = 1]\right| \leq \mathsf{negl}(\lambda)$$

where $\tilde{C}_0 \leftarrow \mathsf{iO}(1^\lambda, C_0)$, $\tilde{C}_1 \leftarrow \mathsf{iO}(1^\lambda, C_1)$.

2.9 Obfuscation for Compute-and-Compare Program

We briefly recall a class of programs called compute-and-compare (CC) programs and their obfuscation, referred to as *compute-and-compare obfuscation* [28,17].

Definition 6 (Compute-and-Compare (CC) Program). *Given a function $f : \{0,1\}^{v(\lambda)} \to \{0,1\}^{w(\lambda)}$, a target value $y \in \{0,1\}^{w(\lambda)}$, the following program is called a compute-and-compare program.*

$$P_{f,y}(x) = \begin{cases} 1, & \text{if } f(x) = y \\ 0, & \text{otherwise} \end{cases}$$

A distribution $\mathcal{D}_\lambda$ over f and y for CC programs is called *sub-exponentially unpredictable* if, for any QPT adversary, given the auxiliary information aux and oracle access to f, the adversary can predict the target value y only with sub-exponential probability.

We note that the CC programs to be obfuscated here are required to be *evasive* functions, as in [28], meaning informally that they output 0 with overwhelming probability when the input x is chosen at random.

Definition 7 (Compute-and-Compare Obfuscation [28,12]). *Let CCObf be a PPT algorithm, which takes as input a compute-and-compare program $P_{f,y}$, a security parameter λ, and outputs a classical program $\tilde{P}_{f,y}$. Let $\mathcal{D}_\lambda$ be a sub-exponentially unpredictable distribution parameterized with λ, and sample $(P_{f,y}, \mathsf{aux}) \leftarrow \mathcal{D}_\lambda$. We say that CCObf is an obfuscation scheme for a compute-and-compare program $P_{f,y}$ if it satisfies the following properties:*

Correctness. *There exists a negligible function $\mathsf{negl}(\lambda)$ such that, for $P_{f,y}$,*

$$\Pr[\forall x : P_{f,y}(x) = \tilde{P}_{f,y}(x) \mid \tilde{P}_{f,y} \leftarrow \mathsf{CCObf}(1^\lambda, P_{f,y})] \geq 1 - \mathsf{negl}(\lambda).$$

Security. *For every QPT adversary $\mathcal{A}$, there exists a PPT simulator CCSim that outputs a program P' satisfying $P'(x) = 0$ for all x. $\mathcal{A}$ can distinguish between the following two distributions with at most $\mathsf{negl}(\lambda)$ advantage:*

$$\left|\Pr[\mathcal{A}(\mathsf{CCObf}(1^\lambda, P_{f,y}), \mathsf{aux}) = 1] - \Pr[\mathcal{A}(\mathsf{CCSim}(1^\lambda, |f|, |y|), \mathsf{aux}) = 1]\right| \leq \mathsf{negl}(\lambda).$$

Theorem 2 ([28]). *Assuming the hardness of LWE, there exists a compute-and-compare obfuscation scheme for any class of CC programs drawn from sub-exponentially unpredictable distributions.*

This guarantees that $\mathsf{CCObf}(1^\lambda, P_{f,y})$ leaks no partial information about (f, y).

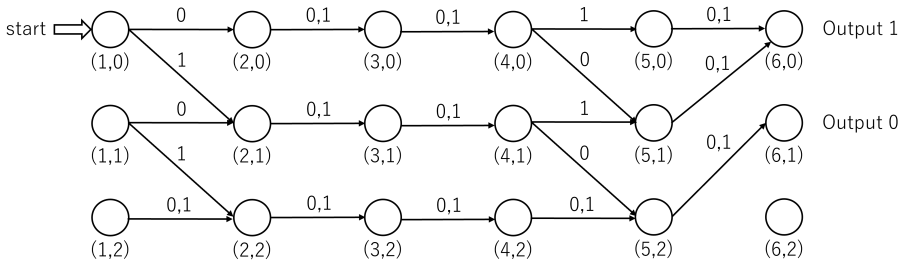


Fig. 2. An example of a branching program. If the input is 10001, the output is 0.

2.10 Branching Programs

We recall branching programs (e.g., Fig. 2) used to represent compute-and-compare programs in [28,12], along with their obfuscation. A branching program is a directed acyclic graph in which every vertex has exactly two outgoing edges, one labeled 1 and the other labeled 0. The graph has a designated start node. To perform a computation, we begin at the start state, and, at each step, follow the edge corresponding to the bit of the input bit string labeling the current node. When a node labeled with an output value is reached, the computation outputs that value.

General branching programs representing CC programs can be obfuscated via the compute-and-compare obfuscation CCObf (Definition 7).

Theorem 3 (Obfuscation for Compute-and-Compare Branching Programs [28,12]). *Assuming the hardness of LWE, there exists compute-and-compare obfuscation* CCObf *for compute-and-compare branching programs.*

2.11 Hidden Subspace States

A subspace state is $A = \sum_{v \in A} |v\rangle$ where A is a subspace of the vector space $\mathbb{F}_2^n$. We will overload the notation by also writing A and $A^\perp$ to denote the corresponding membership-checking programs for A and its orthogonal complement $A^\perp$. In other words, $\mathsf{iO}(A)$ (resp. $\mathsf{iO}(A^\perp)$) checks whether its input belongs to A (resp. $A^\perp$), outputting 1 if so and 0 otherwise.

The following theorem states that any QPT adversary, given a single copy of $|A\rangle$ and membership-checking programs $\big(\mathsf{iO}(A), \mathsf{iO}(A^\perp)\big)$, can clone $|A\rangle$ with only negligible probability.

Theorem 4 ($1 \to 2$ Unclonability [32]). *Consider the following game between a challenger and an adversary $\mathcal{A}$.*

$\underline{\mathsf{Exp}_{\mathcal{A}}(1^\lambda):}$

1. *The challenger $\mathcal{C}$ samples a random subspace $A \subseteq \mathbb{F}_2^\lambda$ of dimension $\lambda/2$.*

2. *$\mathcal{C}$ sends $|A\rangle$, $\mathsf{iO}(A)$, $\mathsf{iO}(A^\perp)$ to $\mathcal{A}$.*
3. *$\mathcal{A}$ prepares and sends a (entangled) bipartite register R_1, R_2 to $\mathcal{C}$.*
4. *$\mathcal{C}$ applies the projective measurement $\{|A\rangle\langle A|, I - |A\rangle\langle A|\}$ to both R_1, and R_2, and outputs 1 if both measurements succeed. Otherwise, it outputs 0.*

Then, for any QPT adversary $\mathcal{A}$, we have $\Pr[\mathsf{Exp}_{\mathcal{A}}(1^\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

The next theorem ensures that any QPT adversary, given $\left(|A\rangle, \mathsf{iO}(A), \mathsf{iO}(A^\perp)\right)$, can output both vectors $v \in A$ and $w \in A^\perp$ only with negligible probability.

Theorem 5 (Direct Product Hardness [24]). *Consider the following game between a challenger and an adversary $\mathcal{A}$.*

$\underline{\mathsf{Exp}_{\mathcal{A}}(1^\lambda)}$

1. *The challenger $\mathcal{C}$ samples a random subspace $A \subseteq \mathbb{F}_2^\lambda$ of dimension $\lambda/2$.*
2. *$\mathcal{C}$ sends $(|A\rangle, \mathsf{iO}(A), \mathsf{iO}(A^\perp))$ to $\mathcal{A}$.*
3. *$\mathcal{A}$ sends two vectors $v, w \in \mathbb{F}_2^\lambda$ to $\mathcal{C}$.*
4. *$\mathcal{C}$ outputs 1 if $(v \in A \setminus \{0^\lambda\} \wedge w \in A^\perp \setminus \{0^\lambda\})$, otherwise, it outputs 0.*

Then, for any QPT adversary $\mathcal{A}$, we have $\Pr[\mathsf{Exp}_{\mathcal{A}}(1^\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

3 Efficiently Realizable OTM Secure against Constant-Depth-Bounded Quantum Adversary

3.1 Basic Construction of OTM

We refer to an adversary who cannot execute quantum circuits of depth greater than d as a d-depth bounded adversary. Assuming d is a constant as in [20], we now construct an OTM protocol as follows:

Construction 8 (OTM with Wiesner States).
$\mathsf{Token.Gen}(1^\lambda, m_0, m_1)$: On input m_0, $m_1 \in \{0,1\}^{\ell_{msg}}$, the sender $\mathcal{S}$ performs:

1. Sample s, $\theta \xleftarrow{\$} \{0,1\}^n$ and f_b, $y_b \xleftarrow{\$} \mathcal{D}_\lambda$[4] for $b \in \{0,1\}$ where $y_b \in \{0,1\}^{\ell_{tgt}}$.
2. Prepare the quantum state $|s^\theta\rangle = H^\theta |s\rangle$.
3. Prepare the program PMsg_b (Algorithm 2) using s, θ, f_b, y_b and m_b.
4. For each $b \in \{0,1\}$, compile $\mathsf{OPMsg}_b \leftarrow \mathsf{CCObf}(\mathsf{PMsg}_b)$[5].
5. Send $|\mathsf{tk}\rangle = (|s^\theta\rangle, \mathsf{OPMsg}_0, \mathsf{OPMsg}_1)$ to the receiver.

※ Here, PMsg_b denotes the code of Algorithm 2, and OPMsg_b denotes the obfuscated code of PMsg_b where PMsg_b checks the measurement outcome s' obtained in Step 2 of $\mathsf{Token.Eval}$ and outputs m_b if s' is valid.

[4] A distribution $\mathcal{D}_\lambda$ over f_b and y_b is sub-exponentially unpredictable.
[5] More precisely, CCObf here should be an obfuscation scheme for a multi-bit output version, referred to as MBCC. We use the notation CCObf for simplicity, and refer to §3.3 for details.

Algorithm 2 $\mathsf{PMsg}_b(x)$

Hardcoded: $s, \theta \in \{0,1\}^n$, $f_b : \{0,1\}^n \to \{0,1\}^{\ell_{tgt}}$, $y_b \in \{0,1\}^{\ell_{tgt}}, m_b \in \{0,1\}^{\ell_{msg}}$.
Input: $x \in \{0,1\}^n$

Compute $f_b(x) = \begin{cases} y_b, & \text{if } x[i] = s[i]\ \forall i \in [n] \text{ with } \theta[i] = b, \\ \bot, & \text{otherwise} \end{cases}$
if $f_b(x) = y_b$ **then**
  return m_b
else
  return 0
end if

$\mathsf{Token.Eval}(b, |\mathsf{tk}\rangle)$: On input $b \in \{0,1\}$ and a token $|\mathsf{tk}\rangle$ from the sender $\mathcal{S}$, the receiver $\mathcal{R}$ performs:

1. Parse $|\mathsf{tk}\rangle = (|s^\theta\rangle, \mathsf{OPMsg}_0, \mathsf{OPMsg}_1)$.
2. Measure $(H^b)^{\otimes n} |s^\theta\rangle$ in the computational basis to obtain s'.
3. Output $m_b = \mathsf{OPMsg}_b(s')$.

We show that Construction 8 is correct and secure against $\mathcal{O}(1)$-depth bounded adversaries.

Lemma 2 (Correctness and Security of Construction 8). *Assuming the quantum hardness of LWE, Construction 8 is an OTM secure against $\mathcal{O}(1)$-depth bounded, if all measurements are error-free ($\epsilon = 0$).*

Proof (Sketch).
Correctness. If $\mathcal{R}$ chooses $b = 0$, the measurement outcome s' matches s at all positions i where $\theta[i] = 0$ since those positions are encoded in the computational basis. When f_0 receives such s', f_0 returns y_0, leading OPMsg_0 to correctly output m_0. Similarly if $b = 1$, then $s'[i] = s[i]$ holds for all i such that $\theta[i] = 1$. In this case, $f_1(s') = y_1$, and hence $\mathsf{OPMsg}_1(s') = m_1$ as desired.

Security. We assume that $\mathcal{R}$ has noisy intermediate-scale quantum (NISQ)-level capability, modeled as an $\mathcal{O}(1)$-depth bounded adversary whose gates have fan-in at most 2 and fan-out 1. Each OPMsg_b checks $n/2$ input bits and thus the execution of this obfuscated program requires depth at least $c \log_2(n/2)$ for some constant $c > 1$, i.e., $\mathcal{O}(\log n)$, to determine its output (Fig. 3). This depth cannot be reduced since all $n/2$ bits affect the output, and the obfuscation prevents $\mathcal{R}$ from exploiting shortcuts. If $c \log_2(n/2) > d$ for a small constant d, $\mathcal{R}$ cannot execute the full quantum circuit of OPMsg_b. Consequently, $\mathcal{R}$ cannot perform a rewinding attack (see §1.2). A d-depth bounded adversary must measure all quantum states before learning the output of OPMsg_b, obtaining only classical information (Fig. 1). After measuring the Wiesner state $|s^\theta\rangle$ to obtain m_b honestly, $\mathcal{R}$ cannot find an input accepted by OPMsg_{1-b}, as the state is irreversibly collapsed and the number of possible valid inputs is $2^{n/2}$. □

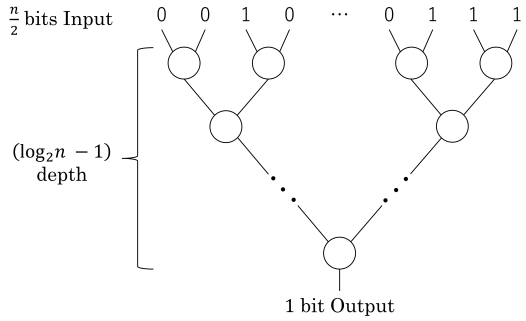


Fig. 3. Illustration of computing a program $P : \{0,1\}^{n/2} \to \{0,1\}$. Each circle represents a single (quantum) gate.

3.2 Upgrading Practicality through Measurement Error Tolerance

So far, we have assumed error-free measurements. In practice, however, measurement errors occur with non-negligible probability at the NISQ level, and even a single error can cause the obfuscated programs or the OTM to fail. Since the security of the OTM relies on the one-time transmission of Wiesner states, the sender cannot retransmit the states in case of error.

To address this, we modify f_b in PMsg_b (Algorithm 2) to accept input strings within a small Hamming distance of the correct input, similar to **PHard** (Algorithm 1). With this modification, f_b outputs the same y_b even if a few measurement bits are flipped. We assume at most c of n bits are flipped, where $c \leq 0.14n$.

Lemma 3 (OTM with Measurement Error Tolerance). *We modify f_b in PMsg_b (Algorithm 2) to f'_b defined as follows:*

$$f'_b(x) = \begin{cases} y_b, & \text{if } \mathsf{HD}(x, t_b) \leq c, \\ \perp, & \text{otherwise} \end{cases} \quad \text{where } t_b[j] = \begin{cases} s[j], & \text{if } \theta[j] = b, \\ *, & \text{otherwise} \end{cases} \quad \forall j \in [n] \qquad (1)$$

with the convention that $\mathsf{HD}(0, *) = \mathsf{HD}(1, *) = 0$.

Then Construction 8 with f_b replaced by f'_b also yields an OTM protocol that tolerates up to c measurement errors.

Proof (Sketch). Correctness of the modified PMsg_b is immediate. For security, assuming $c \leq 0.14n$, and compared with Construction 8, the fraction of inputs x for which $\mathsf{PMsg}_b(x)$ with (1) outputs m_b is roughly[6] at most $(2^{n/2} \cdot 2^{0.29n})/2^n$, which remains negligible in n, and thus $\mathsf{PMsg}_b(x)$ remains an evasive function as required. □

[6] E.g., when $c = 0.025n$, $n = 240$, the more exact fraction is $\approx (2^{n/2} \cdot \sum_{k=0}^{c/2} \binom{n/2}{k})/2^n \approx 2^{-102}$.

3.3 Upgrading the Efficiency of Obfuscation

Now we consider how to concretely and efficiently obfuscate $\mathsf{PMsg}_b(x)$ (Algorithm 2) where $f_b(x)$ is replaced by $f'_b(x)$ (in (1)). To this end, we consider applying the obfuscation technique for multi-bit compute-and-compare (MBCC) programs in [28,12]. Here $\mathsf{PMsg}_b(x)$ corresponds to the MBCC program $P_{f'_b,\, y_b,\, m_b}(x)$ defined as

$$\mathsf{PMsg}_b(x) = P_{f'_b,\, y_b,\, m_b}(x) = \begin{cases} m_b, & \text{if } f'_b(x) = y_b, \\ \bot, & \text{otherwise.} \end{cases} \tag{2}$$

In [28,12], obfuscating the MBCC program $P_{f'_b,\, y_b,\, m_b}(x)$ is further reduced to obfuscating ℓ_{msg} CC programs (see Definitions 6 and 7), each responsible for outputting one bit of $m_b \in \{0,1\}^{\ell_{msg}}$. More specifically, to obfuscate $P_{f'_b,\, y_b,\, m_b}(x)$, it is first represented as a series of CC programs

$$P_{G_1 \circ f'_b,\, y_{b,1}}(x), \ldots, P_{G_i \circ f'_b,\, y_{b,i}}(x), \ldots, P_{G_n \circ f'_b,\, y_{b,\ell_{msg}}}(x), \text{ where, for } i \in [\ell_{msg}], \tag{3}$$

$$P_{G_i \circ f'_b,\, y_{b,i}}(x) = \begin{cases} 1, & \text{if } (G_i \circ f'_b)(x) = y_{b,i} \\ 0, & \text{otherwise} \end{cases}, \quad y_{b,i} = \begin{cases} G_i(y_b), & \text{if } m_b[i] = 1 \\ \overline{G_i(y_b)}, & \text{if } m_b[i] = 0. \end{cases}$$

Here each G_i is a pseudorandom generator (PRG)[7], with $(G_i \circ f'_b)(x) = G_i(f'_b(x))$, and $\overline{G_i(y_b)}$ denoting the bitwise complement. The obfuscated $P_{f'_b, y_b, m_b}(x)$ is obtained by obfuscating the ℓ_{msg} CC programs $P_{G_i \circ f'_b, y_{b,i}}(x)$ in (3), so the computation of each G_i must also be included in the obfuscation. Intuitively, G_i are required to yield multiple random-looking target values $y_{b,i}$ since compute-and-compare obfuscation requires random target values although the underlying function f'_b is fixed here.

In our setting, however, each $G_i \circ f'_b$ in (3) can be replaced by an almost equivalent function $f_{b,i}$ defined in (4) of $\mathsf{PMsg}_b[i]$ (Algorithm 3), where $y_{b,1}, \ldots, y_{b,\ell_{msg}} \in \{0,1\}^{\ell_{tgt}}$ are just sampled uniformly at random without invoking G_i. This removes the need to compute G_i and improves efficiency. Thus PMsg_b in (2) consists of $(\mathsf{PMsg}_b[1], \ldots, \mathsf{PMsg}_b[\ell_{msg}])$.

Next, we discuss efficient obfuscation of each CC program $\mathsf{PMsg}_b[i]$ (Algorithm 3). Following [28,12], the program must first be converted into a small branching program (BP). If this conversion is not possible, one must resort to FHE-based obfuscation of [28], which is significantly less efficient. We note that while [28] required permutation BPs, [12] showed, via a clever trick, that non-permutation BPs can also be obfuscated efficiently.

Thus, the main task is to represent $f_{b,i}(x)$ in (4) as a compact non-permutation BP without relying on heavy machinery such as error-correcting codes for Hamming distance. Since a BP outputs a single bit while $f_{b,i}(x)$ outputs an m-bit

[7] In [28], the LWE-based PRG of [2] is used. This PRG, employing rounding in the Learning With Rounding (LWR) problem, lies in NC^1 and is strongly injective.

Algorithm 3 $\mathsf{PMsg}_b[i](x)$ (where $i \in [\ell_{msg}]$)

Hardcoded: $s, \theta \in \{0,1\}^n$, $f_{b,i} : \{0,1\}^n \to \{0,1\}^{\ell_{tgt}}$, $y_{b,i}, z_{b,i}, v_{b,i} \in \{0,1\}^{\ell_{tgt}}$, $m_b[i] \in \{0,1\}$.
Input: $x \in \{0,1\}^n$
Note: The target value $v_{b,i}$ is predetermined as $v_{b,i} = \begin{cases} y_{b,i}, & \text{if } m_b[i] = 1 \\ z_{b,i}, & \text{if } m_b[i] = 0 \end{cases}$.

1: Compute the following $f_{b,i}(x)$. Here $\overline{y}_{b,i}$ denotes the bitwise complement of $y_{b,i}$.

$$f_{b,i}(x) = \begin{cases} y_{b,i}, & \text{if } \mathsf{HD}(x,\, t_b) \le c, \\ \overline{y}_{b,i}, & \text{otherwise} \end{cases} \text{ where } t_b[j] = \begin{cases} s[j], & \text{if } \theta[j] = b, \\ *, & \text{otherwise} \end{cases} \forall j \in [n] \qquad (4)$$

▷ The output bit equals $m_b[i]$ if $\mathsf{HD}(x, t_b) \le c$, and 0 otherwise
2: **if** $f_{b,i}(x) = v_{b,i}$ [9] **then**
3: **return** 1
4: **else**
5: **return** 0
6: **end if**

string, we require ℓ_{tgt} BPs ($\mathsf{BPMsg}_b[i,1], \ldots, \mathsf{BPMsg}_b[i,\ell_{tgt}]$) to represent $f_{b,i}(x)$, as in [28,12][8]. Consequently, $\mathsf{PMsg}_b(x)$ requires $\ell_{tgt} \times \ell_{msg}$ BPs in total.

We now describe how to construct a compact $\mathsf{BPMsg}_b[i,j]$ that computes the j-th output bit of $f_{b,i}(x)$. In the BP, we start at node $(1,0)$ and read the input $x \in \{0,1\}^n$ sequentially (see Fig. 2). If x is error-free, the computation proceeds along the nodes $(2,0)$, $(3,0), \ldots$, in the top row and terminates at node $(n+1,\, 0)$. If x contains an error, the path moves down by one row. For example, the path becomes $(1,0), (2,1), (3,1), \ldots$, if $x[1]$ is an erroneous bit. In general, if x includes k measurement errors, the computation reaches node $(n+1,\, k)$. At the final node, we check whether $k \le c$, which corresponds to $\mathsf{HD}(x,\, t_b) \le c$.

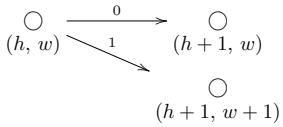
The detailed construction of $\mathsf{BPMsg}_b[i,j]$ is given below.

Construction 9 ($\mathsf{BPMsg}_b[i,j]$)**.** For $i \in [\ell_{msg}]$, $j \in [\ell_{tgt}]$, a branching program $\mathsf{BPMsg}_b[i,j]$ is constructed with the following rules.

1. For indices $h \in [n]$, $w \in \{0, \ldots, c+1\}$ and t_b defined in (4), the following rules are applied.
   - When $t_b[h] = 0$:
     - A 0-labeled edge transitions to a node at the w-th row and a 1-labeled edge transitions to a node at the $w+1$-th row. This can be

[8] The obfuscated equality check in Step 2 of $\mathsf{PMsg}_b[i]$ can be computed just by combining the matrices corresponding to the outputs of these ℓ_{tgt} BPs, following [28,12].
[9] Here $y_{b,i}, z_{b,i} \in \{0,1\}^{\ell_{tgt}}$ are chosen uniformly at random, and with overwhelming probability $y_{b,i}$, $\overline{y}_{b,i}$, $z_{b,i}$ are all distinct when ℓ_{tgt} is large enough. Also, we note that when $v_{b,i} = z_{b,i}$ (i.e., $m_b[i] = 0$), $\mathsf{PMsg}_b[i](x)$ always evaluates to 0.

illustrated pictorially as follows.

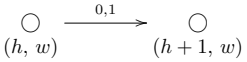


(5)

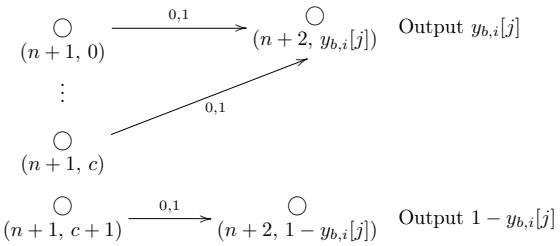
- We note, for example, that the nodes such as $(h+1, w)$ and $(h+1, w+1)$ are referred to as being in the $(h+1)$-th column of the BP[10].

– When $t_b[h] = 1$:
- A 1-labeled edge transitions to a node at the w-th row, and a 0-labeled edge transitions to a node at the $w+1$-th row. This is identical to (5), except that the edge labels 0 and 1 are swapped.

– When $t_b[h] = *$:
- A 0-labeled edge and a 1-labeled edge transition to a node at the same row. This can be illustrated pictorially as follows.



2. When $h = n+1$, the following rule is applied.
   – the first $(c+1)$ nodes transition to the node labeled "Output $y_{b,i}[j]$", and the $(c+2)$-th node transitions to the node labeled "Output $1 - y_{b,i}[j]$". This can be illustrated pictorially as follows.



(6)

We note that the correctness of the obfuscated $\mathsf{PMsg}_b(x)$ follows from the correctness of each $\mathsf{BPMsg}_b[i, j]$.

Lemma 4 (Correctness of Construction 9). *Let $\mathsf{BPMsg}_b[i, j]$ be the BP generated by Construction 9. For every input $x \in \{0,1\}^n$, $\mathsf{BPMsg}_b[i, j](x)$ outputs the j-th bit of $f_{b,i}(x)$ in (4).*

Proof. $\mathsf{BPMsg}_b[i, j]$ compares the input x with the secret string t_b in (4) bit by bit. Upon a mismatch, $\mathsf{BPMsg}_b[i, j]$ transitions from a node (h, w) (i.e., located at column h, row w) to the node $(h+1, w+1)$. Since the initial state is the

[10] We follow the notation conventions in [28].

node $(h, w) = (1, 0)$, reaching a node $(n+1, k)$ where $k \leq c$, indicates that the Hamming distance between the input x and t_b is at most c. Therefore, as in (6), by merging the first $(c+1)$ nodes in the $(n+1)$-th column into the node $(n+2, y_{b,i}[j])$ and redirecting the transition from the node $(n+1, c+1)$ to the node $(n+2, 1-y_{b,i}[j])$, the output of $\mathsf{BPMsg}_b[i,j]$ becomes $y_{b,i}[j]$ or $\overline{y_{b,i}[j]} = 1 - y_{b,i}[j]$ accordingly. This exactly matches the j-th output bit of $f_{b,i}(x)$, establishing correctness. □

We state the following lemma, which shows that there exists a compute-and-compare obfuscation for $\mathsf{BPMsg}_b[i,j]$ for $i \in [\ell_{msg}]$, $j \in [\ell_{tgt}]$.

Lemma 5 (Obfuscation for $\mathsf{BPMsg}_b[i,j]$). *Under the LWE assumption, there exists a compute-and-compare obfuscation for* $(\mathsf{BPMsg}_b[i,1], \ldots, \mathsf{BPMsg}_b[i,\ell_{tgt}])$.

Proof (Sketch). The BPs $\{\mathsf{BPMsg}_b[i,j]\}_{j\in[\ell_{tgt}]}$ generated by Construction 9 are the BPs of a CC program, and similarly to the proof sketch of Lemma 3, the fraction of inputs x for which $\{\mathsf{BPMsg}_b[i,j]\}_{j\in[\ell_{tgt}]}$ outputs $m_b[i] \in \{0,1\}$ is negligible in n, as required. Furthermore, in our construction, $y_{b,i}$ and $z_{b,i}$ in $\mathsf{PMsg}_b[i](x)$ (Algorithm 3) can be, and indeed are chosen at random, so the target values $v_{b,i}$ (which is equal to either $y_{b,i}$ or $z_{b,i}$) are sampled from an unpredictable distribution $\mathcal{D}_\lambda$. Hence, by Theorems 2 and 3, there exists a compute-and-compare obfuscation for $\{\mathsf{BPMsg}_b[i,j]\}_{j\in[\ell_{tgt}]}$. □

3.4 Main Construction

We now combine the building blocks to construct an efficiently realizable OTM protocol $(\mathsf{Token.Gen}, \mathsf{Token.Eval})$. The protocol is executed between a sender $\mathcal{S}$ and a receiver $\mathcal{R}$, and is secure against $\mathcal{O}(1)$-depth bounded quantum circuits.

Construction 10 (Efficiently Realizable OTM).
$\mathsf{Token.Gen}(1^\lambda, m_0, m_1)$: On input $m_0, m_1 \in \{0,1\}^{\ell_{msg}}$, the sender $\mathcal{S}$ performs:

1. Sample $s, \theta \xleftarrow{\$} \{0,1\}^n$ with $\mathsf{HW}(\theta) = \frac{n}{2}$.
2. Prepare $|s^\theta\rangle = H^\theta |s\rangle$.
3. Choose appropriate $c \leftarrow \mathbb{N}$ where $c \ll n$. Sample $f_{b,i}, v_{b,i} \xleftarrow{\$} \mathcal{D}_\lambda$ where $y_{b,i}, z_{b,i} \xleftarrow{\$} \{0,1\}^{\ell_{tgt}}$ for $b \in \{0,1\}$ and $i \in [\ell_{msg}]$ (see (4) of $\mathsf{PMsg}_b[i](x)$ in Algorithm 3 corresponding to $\mathsf{BPMsg}_b[i,j]$).
4. Compile $\mathsf{OBPMsg}_b[i,j] \leftarrow \mathsf{CCObf}(\mathsf{BPMsg}_b[i,j])$ for $b \in \{0,1\}$, $i \in [\ell_{msg}]$, $j \in [\ell_{tgt}]$ where $\mathsf{BPMsg}_b[i,j]$ is given in Construction 9.
5. Send $|\mathsf{tk}\rangle := (|s^\theta\rangle, \mathsf{OBPMsg}_0 = \{\mathsf{OBPMsg}_0[i,j]\}_{i\in[\ell_{msg}],\, j\in[\ell_{tgt}]}, \mathsf{OBPMsg}_1 = \{\mathsf{OBPMsg}_1[i,j]\}_{i\in[\ell_{msg}],\, j\in[\ell_{tgt}]})$ to the receiver.

$\mathsf{Token.Eval}(b, |\mathsf{tk}\rangle)$: On input $b \in \{0,1\}$ and a token $|\mathsf{tk}\rangle$ from the sender $\mathcal{S}$, the receiver $\mathcal{R}$ performs:

1. Parse $|\mathsf{tk}\rangle$ into $(|s^\theta\rangle, \mathsf{OBPMsg}_0, \mathsf{OBPMsg}_1)$.
2. Measure $(H^b)^{\otimes n} |s^\theta\rangle$ in the computational basis to obtain s'.

3. Compute $m_b \leftarrow \mathsf{OBPMsg}_b(s')$.

Theorem 6 (Correctness and Security of Construction 10). *Assuming the quantum hardness of LWE, Construction 10 is an OTM protocol secure against $\mathcal{O}(1)$-depth bounded adversaries, if the measurement error rate is up to $\epsilon \leq 0.14$.*

Proof (Sketch).
Correctness. Construction 10 corresponds to the error-tolerant version of Construction 8, differing only in the use of branching programs BPMsg_b. Lemma 4 shows that this difference in implementation does not affect correctness. Moreover, Lemmas 2 and 3 establish the correctness of the error-tolerant version of Construction 8. Hence, the correctness of Construction 10 follows.

Security. Lemma 5 shows that each branching program $\{\mathsf{BPMsg}_b[i,j]\}_{i\in[\ell_{msg}],j\in[\ell_{tgt}]}$ can be securely obfuscated using compute-and-compare obfuscation. It follows that BPMsg_b as a whole can also be securely obfuscated against constant-depth-bounded adversaries as in Construction 8, since it is a collection of such branching programs. Similarly, Lemmas 2 and 3 guarantee the security of the error-tolerant version of Construction 8; thus, the security of Construction 10 also follows. □

4 Limitation of Construction 10 : Bit-Flipping Attack

We have shown that Wiesner states combined with compute-and-compare obfuscation yield OTMs secure against $\mathcal{O}(1)$-depth bounded adversaries. A natural question then arises:

> *"Can one construct OTMs secure against non-constant-depth-bounded adversaries, still using Wiesner states and compute-and-compare obfuscation, by forcing the adversary to employ deeper quantum circuits via the sequential composition of obfuscated CC programs?"*

We conjecture that the answer is *no*. The relevant attack (or observation) is similar to that in [21], [3, §2.1], which present an adaptive attack on Wiesner states using either a verifying oracle or obfuscated programs that check their validity.

As an illustrative insecure construction, consider an OTM against logarithmic-depth bounded adversaries. In addition to PMsg_0 and PMsg_1, we introduce $\mathsf{PCom}_b^{(i)}$ for $i \in [k]$ and $b \in \{0,1\}$, where each $\mathsf{PCom}_b^{(i)}$ is an obfuscated CC program. $\mathsf{PCom}_b^{(1)}$ takes $x_b^{(0)} \in \{0,1\}^n$ as input and outputs $x_b^{(1)}$ if $x_b^{(0)}$ is obtained by measuring $(H^b)^{\otimes n}\,|s^\theta\rangle$ in the computational basis, and outputs $\perp$ otherwise. More generally, $\mathsf{PCom}_b^{(i)}$ takes $x_b^{(i-1)}$ as input and outputs $x_b^{(i)}$ only if $x_b^{(i-1)}$ is a valid output of $\mathsf{PCom}_b^{(i-1)}$. Finally, PMsg_b is modified to output m_b upon accepting a valid $x_b^{(k)}$.

At first glance, this construction appears secure against, e.g., $\mathcal{O}(\log n)$-depth bounded adversaries, since obtaining m_b seems to require evaluating a quantum

circuit of depth $\Omega(k \log n)$, i.e., $m_b \leftarrow \mathsf{PMsg}_b \circ \mathsf{PCom}_b^{(k)} \circ \cdots \circ \mathsf{PCom}_b^{(1)}(x_b^{(0)})$, under the assumption that each $\mathsf{PCom}_b^{(i)}$ can be computed by logarithmic-depth circuits. However, a logarithmic-depth bounded adversary can clone the Wiesner state $|s^\theta\rangle$ as follows. Without loss of generality, we can assume that some CC program[11] in $\mathsf{PCom}_b^{(1)}$ outputs 1 on a valid encoding of $|s^\theta\rangle$. Then, to learn the i-th qubit of $|s^\theta\rangle$, the adversary applies a Pauli-X gate to the i-th qubit, feeds $|s^\theta\rangle$ into the CC program coherently, and measures the output without disturbing $|s^\theta\rangle$. If the result is 1, the i-th qubit is $|+\rangle$ or $|-\rangle$; otherwise, it is $|0\rangle$ or $|1\rangle$. Repeating this process for all n qubits of $|s^\theta\rangle$ reveals θ and $|s^\theta\rangle$, enabling cloning.

5 OTM Secure against $\mathcal{O}(\lambda^\gamma)$-depth Bounded Quantum Adversary with iO

We construct an OTM against $\mathcal{O}(\lambda^\gamma)$-depth bounded quantum adversaries in the QROM. This construction is partly inspired by [11, §7] and [19].

Instead of Wiesner states, we employ a *hidden subspace state* $|T_0(A_{\mathsf{Can}})\rangle = \sum_{v \in A_{\mathsf{Can}}} |T_0(v)\rangle$ where A_{Can} denotes the *canonical* $\lambda/2$ subspace of $\mathbb{F}_2^\lambda$ defined as $\mathsf{Span}(e_1, e_2, \ldots, e_{\lambda/2})$ and $e_i \in \mathbb{F}_2^\lambda$ is the standard basis vector with a 1 in the i-th coordinate and 0 in all others. The map T_0 is a full rank linear map and $T_0(A_{\mathsf{Can}})$ can be regarded as a rerandomized hidden subspace.

As also observed in [4], unlike Wiesner states, $|T_0(A_{\mathsf{Can}})\rangle$ is immune to bit-flipping attacks (§4). Intuitively, $T_0(A_{\mathsf{Can}})$ consists of $2^{n/2}$ random vectors over $\mathbb{F}_2^n$. Therefore, for any $v \in T_0(A_{\mathsf{Can}})$, the vector v' obtained by flipping a single qubit of v belongs to $T_0(A_{\mathsf{Can}})$ only with exponentially small probability.

5.1 Construction

We present an OTM construction, building on post-quantum indistinguishability obfuscation (iO) for all polynomial-size classical circuits.

$\mathsf{SampleFullRank}$ is a PPT algorithm that takes a security parameter λ and a seed as inputs, and outputs a full rank linear map $T : \mathbb{F}_2^\lambda \to \mathbb{F}_2^\lambda$. F is a post-quantum *pseudorandom function* (PRF) that takes a key and an input, and outputs the function value. Here $n = \lambda$ and m is polynomial in λ. $\mathsf{PReRand}$ is a rerandomization algorithm that takes as input an id, a vector, and a chosen bit. If $\mathsf{PReRand}$ can extract a vector in A_{Can} from the input vector, the input is considered valid, and $\mathsf{PReRand}$ outputs the next rerandomized vector along with the next rerandomized id. PMem is a memory algorithm that also takes an id, a vector, and a chosen bit as input. PMem outputs the OTM message if and only if these inputs have been rerandomized by $\mathsf{PReRand}$ exactly k times. The depth parameter k is set such that implementing $\mathsf{OPReRand}$ sequentially k times requires a quantum circuit deeper than the adversary's depth bound $\mathcal{O}(\lambda^\gamma)$.

[11] This corresponds to the obfuscated $\mathsf{PMsg}_b[i](x)$ (Algorithm 3) with $m_b[i] = 1$.

Construction 11 (Feasibility Construction of OTM).
$\mathsf{Token.Gen}(1^\lambda, m_0, m_1)$: The sender $\mathcal{S}$ performs the following steps:

1. Sample $K_0, K_1 \leftarrow F.\mathsf{Setup}(1^\lambda)$, $id_0 \leftarrow \{0,1\}^\lambda$.
2. Set $id_k = \underbrace{F(K_1, (F(K_1, (\ldots, F(K_1, id_0)))))}_{F(K_1,\cdot) \text{ applied } k \text{ times}}$.
3. Set $T_0 = \mathsf{SampleFullRank}(1^\lambda; F(K_0, id_0))$, $T_k = \mathsf{SampleFullRank}(1^\lambda; F(K_0, id_k))$.
4. Set $|\psi\rangle = \sum_{v \in A_{\mathsf{Can}}} |T_0(v)\rangle$.
5. Compile $\mathsf{OPReRand} \leftarrow \mathsf{iO}(\mathsf{PReRand})$ where $\mathsf{PReRand}$ is in Algorithm 4.
6. Compile $\mathsf{OPMem} \leftarrow \mathsf{iO}(\mathsf{PMem})$ where PMem is in Algorithm 5.
7. Send the token $|\mathsf{tk}\rangle := (id_0, k, |\psi\rangle, \mathsf{OPReRand}, \mathsf{OPMem})$ to the receiver.

$\mathsf{Token.Eval}(|\mathsf{tk}\rangle)$: The receiver $\mathcal{R}$, upon receiving the token $|\mathsf{tk}\rangle$ from $\mathcal{S}$, selects an input $b \in \{0,1\}$ and executes the following steps:

1. Parse the token $(id_0, k, |\psi\rangle, \mathsf{OPReRand}, \mathsf{OPMem})$.
2. Measure $(H^b)^{\otimes n} |\psi\rangle$ in the computational basis to obtain x_0[12]
3. For $i \in [k]$, run $(id_i, x_i) \leftarrow \mathsf{OPReRand}(id_{i-1}, x_{i-1}, b)$.
4. Output $m_b \leftarrow \mathsf{OPMem}(id_k, x_k, b) \oplus \mathcal{RO}(b \,\|\, x_k)$.

Algorithm 4 $\mathsf{PReRand}(id, v, b)$

Hardcoded : K_0, K_1
If $v = 0^n$, output $\perp$ and halt
$id' = F(K_1, id)$ ▷ Generating subsequent id_{i+1} from id_i
$T_{\mathsf{cur}} = \mathsf{SampleFullRank}(1^\lambda; F(K_0, id))$, $T_{\mathsf{nxt}} = \mathsf{SampleFullRank}(1^\lambda; F(K_0, id'))$
$\omega = T_{\mathsf{cur}}^{-1}(v)$ if $b = 0$; otherwise, $\omega = T_{\mathsf{cur}}^{\top}(v)$
If $b = 0$ and $\omega \in A_{\mathsf{Can}}$, output $T_{\mathsf{nxt}}(\omega)$, id' and halt
If $b = 1$ and $\omega \in A_{\mathsf{Can}}^{\perp}$, output $(T_{\mathsf{nxt}}^{\top})^{-1}(\omega)$, id' and halt
Otherwise, output $\perp$

Algorithm 5 $\mathsf{PMem}(id, v, b)$

Hardcoded : id_k, T_k, m_0, m_1
If $v = 0^n$ or $id \neq id_k$, output $\perp$ and halt
$\omega = T_k^{-1}(v)$ if $b = 0$; otherwise, $\omega = T_k^{\top}(v)$
If $b = 0$ and $\omega \in A_{\mathsf{Can}}$, output $m_0 \oplus \mathcal{RO}(0 \,\|\, v)$ and halt
If $b = 1$ and $\omega \in A_{\mathsf{Can}}^{\perp}$, output $m_1 \oplus \mathcal{RO}(1 \,\|\, v)$ and halt
Otherwise, output $\perp$

The correctness of Construction 11 is established as follows.

[12] We note that the probability that $x_0 = 0^n$ is negligible (namely $1/2^{n/2}$).

Theorem 7 (Correctness). *Construction 11 is correct.*

Proof. First, since A_{Can} is a subspace state and T_0 is a bijection, $T_0(A_{\mathsf{Can}}) = A'$ is also a subspace state where $A' = \{T_0(w) \mid w \in A_{\mathsf{Can}}\}$. In Step 2 of $\mathsf{Token.Eval}$, when $b = 0$, the receiver $\mathcal{R}$ obtains a measurement outcome vector $x_0 := T_0(v_0) \in T_0(A_{\mathsf{Can}})$. Thus, in $\mathsf{PReRand}$ (Algorithm 4), we have $\omega = T_{\mathsf{cur}}^{-1}(x_0) = T_{\mathsf{cur}}^{-1}(T_0(v_0)) = v_0 \in A_{\mathsf{Can}}$ since $T_0 = T_{\mathsf{cur}}$. When $b = 1$, $\mathcal{R}$ obtains $x_0 \in (A')^{\perp}$[13]. Moreover, as shown in [11, §7.1], $x_0 \in (A')^{\perp}$ is equivalent to $T_0^{\top}(x_0) \in A_{\mathsf{Can}}^{\perp}$ (i.e., $x_0 \in (T_0^{\top})^{-1}(A_{\mathsf{Can}}^{\perp})$). Thus, in $\mathsf{PReRand}$, we have $\omega = T_{\mathsf{cur}}^{\top}(x_0) = T_0^{\top}(x_0) \in A_{\mathsf{Can}}^{\perp}$, again since $T_0 = T_{\mathsf{cur}}$. Hence, if the input (id, v, b) to $\mathsf{PReRand}$ is valid, $\mathsf{PReRand}$ produces another valid input in the subsequent step. By iteratively executing the remaining steps of $\mathsf{Token.Eval}$, $\mathcal{R}$ eventually recovers m_b correctly. □

5.2 Measurement Error Tolerance

This OTM can be made immune to measurement errors. We assume up to $c \in \mathbb{N}$ errors, and apply suitable classical error-correcting codes capable of correcting up to c errors.

- $\mathsf{ECC.Enc}$: Takes a bit string x and outputs a codeword C.
- $\mathsf{ECC.Dec}$: Takes a codeword C and outputs the decoded bit string x if C contains at most c errors.

The sender $\mathcal{S}$ replaces $|\psi\rangle = \sum_{v \in A_{\mathsf{Can}}} |T_0(v)\rangle$ with $\sum_{v \in A_{\mathsf{Can}}} |\mathsf{ECC.Enc}(T_0(v))\rangle$ in $\mathsf{Token.Gen}$, and adds $\mathsf{ECC.Dec}$ to $|\mathsf{tk}\rangle$. The receiver $\mathcal{R}$ then computes $x_0 \leftarrow \mathsf{ECC.Dec}(C)$ where C is the measurement outcome of $|\psi\rangle$. It is straightforward to verify that the construction remains correct. When c is sufficiently small relative to n, security follows from Theorem 8.

5.3 Security Analysis

We analyze the security of Construction 11. First, we consider the *rewinding attack* (see §1.2). When the adversary $\mathcal{A}$ is $\mathcal{O}(\lambda^{\gamma})$-depth bounded, simply setting $k = \Omega(\lambda^{\gamma+1})$ prevents $\mathcal{A}$ from outputting m_b without any intermediate measurements, i.e., it prevents $\mathcal{A}$ from relying solely on gentle measurements of $\mathcal{A}$'s registers and subsequently rewinding the computation on $|T_0(v)\rangle$.

Second, we consider the *bit-flipping attack* (see §4). Suppose that $\mathcal{A}$ attempts to guess $|T(A_{\mathsf{Can}})\rangle$ using $\mathsf{PReRand}$ (Algorithm 4). Here, $\mathsf{PReRand}$ can be used as $\mathsf{iO}(A_{\mathsf{Can}})$ since both return a fixed value when the input is a certain valid *subspace state*. However, if $\mathcal{A}$ can clone the *subspace state*, it contradicts Theorem 4. Hence, the probability that $\mathcal{A}$ succeeds in the bit-flipping attack is negligible.

We now provide a more formal security proof, reducing the security of our construction to the direct product hardness (Theorem 5). That is, we show that if $\mathcal{A}$ can obtain both m_0 and m_1 from the OTM, then we can construct a reduction that uses $\mathcal{A}$ to break this hardness assumption.

[13] This property originates from the subspace-state quantum money scheme [1].

Theorem 8 (Security). *Construction 11 yields an OTM protocol secure against $\mathcal{O}(\lambda^\gamma)$-depth bounded adversaries in the QROM.*

We describe the security through a sequence of hybrids.

Hyb_0 : This hybrid is identical to $\mathsf{OTM\text{-}Game}_{\mathcal{A}}(1^\lambda)$ (Definition 3) where ($\mathsf{Token.Gen}$, $\overline{\mathsf{Token.Eval}}$) are as defined in Construction 11.

Hyb_1 : Sample a random subspace $A^* \subseteq \mathbb{F}_2^\lambda$ and prepare the state $|A^*\rangle$. Define the classical program as $\mathsf{P}_0 \leftarrow \mathsf{iO}(A^*)$ and $\mathsf{P}_1 \leftarrow \mathsf{iO}((A^*)^\perp)$. Set $\mathsf{OPReRand} \leftarrow \mathsf{iO}(\mathsf{PReRand}')$, where $\mathsf{PReRand}'$ is Algorithm 6. Set $\mathsf{OPMem} \leftarrow \mathsf{iO}(\mathsf{PMem}')$, where PMem' is Algorithm 7. The modifications in this hybrids are marked in red.

Algorithm 6 $\mathsf{PReRand}'(id,\, v,\, b)$

Hardcoded : K_0, K_1, $\mathsf{P}_0, \mathsf{P}_1$

1: If $v = 0^n$, output $\perp$ and halt
2: $id' = F(K_1,\, id)$
3: $T_{\mathsf{cur}} = \mathsf{SampleFullRank}(1^\lambda; F(K_0,\, id))$, $T_{\mathsf{nxt}} = \mathsf{SampleFullRank}(1^\lambda; F(K_0,\, id'))$
4: $\omega = T_{\mathsf{cur}}^{-1}(v)$ if $b = 0$; otherwise, $\omega = T_{\mathsf{cur}}^{\top}(v)$
5: If $b = 0$ and $\mathsf{P}_0(\omega) = 1$, output $T_{\mathsf{nxt}}(\omega)$, id' and halt
6: If $b = 1$ and $\mathsf{P}_1(\omega) = 1$, output $(T_{\mathsf{nxt}}^{\top})^{-1}(\omega)$, id' and halt
7: Otherwise, output $\perp$ and halt

Algorithm 7 $\mathsf{PMem}'(id,\, v,\, b)$

Hardcoded : id_k, T_k, m_0, m_1, P_0, P_1

1: If $v = 0^n$ or $id \neq id_k$, output $\perp$ and halt
2: $\omega = T_k^{-1}(v)$ if $b = 0$; otherwise, $\omega = T_k^{\top}(v)$
3: If $b = 0$ and $\mathsf{P}_0(\omega) = 1$, output $m_0 \oplus \mathcal{RO}(0 \,\|\, v)$ and halt
4: If $b = 1$ and $\mathsf{P}_1(\omega) = 1$, output $m_1 \oplus \mathcal{RO}(1 \,\|\, v)$ and halt
5: Otherwise, output $\perp$

Finally, we modify $\mathsf{Token.Gen}$ as follows.

1. Sample K_0, $K_1 \leftarrow F.\mathsf{Setup}(1^\lambda)$, $id_0 \leftarrow \{0,1\}^\lambda$.
2. Set $id_k = F(K_1,\, (F(K_1,\, (\ldots, F(K_1,\, id_0)))))$.
3. Set $T_0 = \mathsf{SampleFullRank}(1^\lambda; F(K_0,\, id_0))$, $T_k = \mathsf{SampleFullRank}(1^\lambda; F(K_0,\, id_k))$.
4. Sample a random subspace $A^* \subseteq \mathbb{F}_2^\lambda$, and set $|\psi\rangle = |T_0(A^*)\rangle$.
5. Compile $\mathsf{OPReRand}' \leftarrow \mathsf{iO}(\mathsf{PReRand}')$ where $\mathsf{PReRand}'$ is in Algorithm 4.
6. Compile $\mathsf{OPMem}' \leftarrow \mathsf{iO}(\mathsf{PMem}')$ where PMem' is in Algorithm 5.
7. Send the token $|\mathsf{tk}\rangle := (id_0,\, k,\, |\psi\rangle\,,\, \mathsf{OPReRand}',\, \mathsf{OPMem}')$ to the receiver.

Lemma 6. $\mathsf{Hyb}_0 \approx \mathsf{Hyb}_1$.

Proof (Sketch). In Hyb_1, the adversary $\mathcal{A}$ receives $|T_0(A^*)\rangle$ instead of $|T_0(A_{\mathsf{Can}})\rangle$. Since A^* can be expressed as $A^* = T_0'(A_{\mathsf{Can}}) = \{T_0'(w) \mid w \in A_{\mathsf{Can}}\}$ for a random full rank linear map T_0', we have $|T_0(A^*)\rangle = |T_0T_0'(A_{\mathsf{Can}})\rangle$. By considering that T_0T_0' in Hyb_1 corresponds to T_0 in Hyb_0, we can argue that this substitution does not affect $\mathcal{A}$'s advantage as follows.

First, both $|T_0(A^*)\rangle$ and $|T_0(A_{\mathsf{Can}})\rangle$ appear as random subspace states to $\mathcal{A}$ due to the randomness of T_0.

Second, the subspace membership check $\mathsf{P}_0(\omega)$ in $\mathsf{OPReRand}'$ and OPMem' in Hyb_1 implements $(T_0')^{-1}(\omega) \stackrel{?}{\in} A_{\mathsf{Can}}$, which is functionally equivalent to the subspace membership check $\omega \stackrel{?}{\in} A_{\mathsf{Can}}$ in $\mathsf{OPReRand}$ and OPMem in Hyb_0. Specifically, in Hyb_1, given $v \in T_kT_0'(A_{\mathsf{Can}})$, checking $(T_0')^{-1}(\omega) \stackrel{?}{\in} A_{\mathsf{Can}}$ corresponds to $(T_kT_0')^{-1}(v) \stackrel{?}{\in} A_{\mathsf{Can}}$, whereas, in Hyb_0, given $v \in T_k(A_{\mathsf{Can}})$, checking $\omega \stackrel{?}{\in} A_{\mathsf{Can}}$ corresponds to $T_k^{-1}(v) \stackrel{?}{\in} A_{\mathsf{Can}}$. Hence, the two checks are functionally equivalent. A similar reasoning applies to $\mathsf{P}_1(\omega)$. Therefore, these hybrids are indistinguishable to $\mathcal{A}$ by the security of iO, and we conclude that $\mathsf{Hyb}_0 \approx \mathsf{Hyb}_1$. □

Lemma 7. $\Pr[\mathsf{Hyb}_1 = 1] \leq \mathsf{negl}(\lambda)$.

Proof (Sketch). We construct a reduction $\mathcal{R}$ which, given a single copy of $|A^*\rangle$ along with $\mathsf{iO}(A^*)$, $\mathsf{iO}((A^*)^\perp)$ in the game of Theorem 5, interacts with $\mathcal{A}$ in Hyb_1, while simulating $\mathcal{RO}$. We assume that the depth parameter k is set such that implementing $\mathsf{OPReRand}'$ requires a quantum circuit exceeding the adversary's depth bound $\mathcal{O}(\lambda^\gamma)$, thereby preventing $\mathcal{A}$ from inputting $|T_k(A^*)\rangle$ into OPMem'. Hence, if $\mathcal{A}$ outputs both m_0 and m_1 with non-negligible probability, then by standard QROM techniques such as the compressed oracle technique [31] and the one-way to hiding lemma [26], $\mathcal{R}$ can extract $(0 \,\|\, v)$ and $(1 \,\|\, w)$ from the $\mathcal{RO}$ transcript with non-negligible probability, recovering both $T_k^{-1}(v) \in A^*$ and $T_k^\top(w) \in (A^*)^\perp$ using the hardcoded T_k. This implies that $\mathcal{R}$ breaks the direct product hardness (Theorem 5), leading to a contradiction. □

References

1. Aaronson, S., Christiano, P.: Quantum money from hidden subspaces. In: STOC. pp. 41–60. ACM (2012)
2. Banerjee, A., Peikert, C., Rosen, A.: Pseudorandom functions and lattices. In: Eurocrypt. pp. 719–737. Springer (2012)
3. Bartusek, J., Goyal, V., Khurana, D., Malavolta, G., Raizes, J., Roberts, B.: Software with certified deletion. Cryptology ePrint Archive, Paper 2023/265 (2023)
4. Bartusek, J., Goyal, V., Khurana, D., Malavolta, G., Raizes, J., Roberts, B.: Software with certified deletion. In: Eurocrypt. pp. 85–111. Springer (2024)
5. Behera, A., Sattath, O., Shinar, U.: Noise-tolerant quantum tokens for MAC (2025), https://arxiv.org/abs/2105.05016
6. Ben-David, S., Sattath, O.: Quantum tokens for digital signatures. Quantum **7**, 901 (2023)

7. Bitansky, N., Goldwasser, S., Jain, A., Paneth, O., Vaikuntanathan, V., Waters, B.: Time-lock puzzles from randomized encodings. In: ITCS. pp. 345–356. ACM (2016)
8. Broadbent, A., Gharibian, S., Zhou, H.S.: Towards quantum one-time memories from stateless hardware. Quantum **5**, 429 (2021)
9. Broadbent, A., Gutoski, G., Stebila, D.: Quantum one-time programs. In: CRYPTO. p. 344–360. Springer Berlin Heidelberg (2013)
10. Broadbent, A., Lord, S.: Uncloneable Quantum Encryption via Oracles. In: TQC. Leibniz International Proceedings in Informatics (LIPIcs), vol. 158, pp. 4:1–4:22. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2020)
11. Cakan, A., Goyal, V., Yamakawa, T.: Anonymous public-key quantum money and quantum voting (2024), https://arxiv.org/abs/2411.04482
12. Chen, Y., Vaikuntanathan, V., Wee, H.: GGH15 beyond permutation branching programs: proofs, attacks, and candidates. In: CRYPTO. pp. 577–607. Springer (2018)
13. Coladangelo, A., Liu, J., Liu, Q., Zhandry, M.: Hidden cosets and applications to unclonable cryptography. In: CRYPTO. pp. 556–584. Springer (2021)
14. Culf, E., Vidick, T.: A monogamy-of-entanglement game for subspace coset states. Quantum **6**, 791 (2022)
15. Eldridge, H., Goel, A., Green, M., Jain, A., Zinkus, M.: One-time programs from commodity hardware. In: TCC. pp. 121–150. Springer (2022)
16. Goldwasser, S., Kalai, Y.T., Rothblum, G.N.: One-time programs. In: CRYPTO. p. 39–56. Springer-Verlag (2008)
17. Goyal, R., Koppula, V., Waters, B.: Lockable obfuscation. In: FOCS. pp. 612–621. IEEE (2017)
18. Gunn, S., Movassagh, R.: Quantum one-time protection of any randomized algorithm. In: CRYPTO. pp. 334–349. Springer (2025)
19. Gupte, A., Liu, J., Raizes, J., Roberts, B., Vaikuntanathan, V.: Quantum one-time programs, revisited. In: STOC. pp. 213–221. ACM (2025)
20. Liu, Q.: Depth-Bounded Quantum Cryptography with Applications to One-Time Memory and More. In: ITCS. Leibniz International Proceedings in Informatics, vol. 251, pp. 82:1–82:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2023)
21. Lutomirski, A.: An online attack against wiesner's quantum money (2010), https://arxiv.org/abs/1010.0256
22. NIST: Submission requirements and evaluation criteria for the post-quantum cryptography standardization process (2016)
23. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: STOC. p. 84–93. ACM (2005)
24. Sattath, O., Wyborski, S.: Uncloneable decryptors from quantum copy-protection (2022), https://arxiv.org/abs/2203.05866
25. Stambler, L.: Quantum one-time memories from stateless hardware, random access codes, and simple nonconvex optimization (2025), https://arxiv.org/abs/2501.04168
26. Unruh, D.: Revocable quantum timed-release encryption. Journal of the ACM (JACM) **62**(6), 1–76 (2015)
27. Wang, P., Luan, C.Y., Qiao, M., Um, M., Zhang, J., Wang, Y., Yuan, X., Gu, M., Zhang, J., Kim, K.: Single ion qubit with estimated coherence time exceeding one hour. Nature communications **12**(1), 233 (2021)
28. Wichs, D., Zirdelis, G.: Obfuscating compute-and-compare programs under LWE. In: FOCS. pp. 600–611. IEEE (2017)

29. Wiesner, S.: Conjugate coding. ACM Sigact News **15**(1), 78–88 (1983)
30. Wootters, W.K., Zurek, W.H.: A single quantum cannot be cloned. Nature **299**(5886), 802–803 (1982)
31. Zhandry, M.: How to record quantum queries, and applications to quantum indifferentiability. In: CRYPTO. pp. 239–268. Springer (2019)
32. Zhandry, M.: Quantum lightning never strikes the same state twice. or: quantum money from cryptographic assumptions. Journal of Cryptology **34**(1), 6 (2021)
33. Zhao, L., Choi, J.I., Demirag, D., Butler, K.R.B., Mannan, M., Ayday, E., Clark, J.: One-time programs made practical. In: Financial Cryptography and Data Security. p. 646–666. Springer (2019)